



第1章 Hibernate_day03

今日任务

- 案例一:使用 Hibernate 完成一对多的关联关系映射并操作
- 案例二:使用 Hibernate 完成多对多的关联关系映射并操作

教学导航

教学目标	
教学方法	案例驱动法

案例一：完成一对多的关联关系映射并操作

1.1 案例需求：

1.1.1 需求描述

一个客户对应多个联系人，单独在联系人管理模块中对联系人信息进行维护，功能包括：添加联系人、修改联系人、删除联系人。

添加联系人：添加联系人时指定所属客户，添加信息包括联系人名称、联系电话等

修改联系人：允许修改联系人所属客户、联系人名称、联系人电话等信息

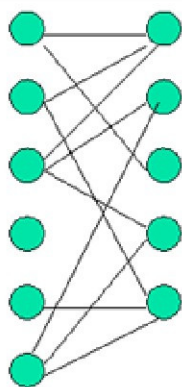
删除联系人：删除客户的同时删除下属的联系人，可以单独删除客户的某个联系人

1.2 相关知识点

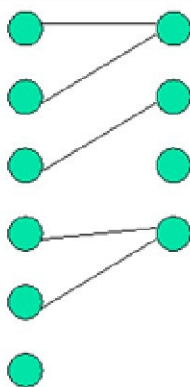
1.2.1 表关系的分析：

Hibernate 框架实现了 ORM 的思想，将关系数据库中表的数据映射成对象，使开发人员把对数据库的操作转化为对对象的操作，Hibernate 的关联关系映射主要包括多表的映射配置、数据的增加、删除等。

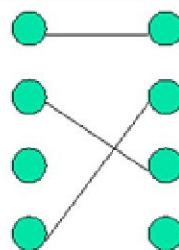
数据库中多表之间存在着三种关系，也就是系统设计中的三种实体关系。如图所示。



多对多
M: N



一对多
1: N



一对一
1: 1

从图可以看出，系统设计的三种实体关系分别为：多对多、一对多和一对一关系。在数据库中，实体表之间的关系映射是采用外键来描述的，具体如下。

1.2.1.1 表与表的三种关系:

【一对多】

建表原则：在多的的一方创建外键指向一的一方的主键：

一对多：在多的的一方创建外键，指向一的一方的主键。

客户表：

ID	NAME	AGE
1	张三	25
2	李四	28

联系人表：

ID	NAME	PHONE	CID
1	王女士	xxx	1
2	刘先生	yyy	1
3	张先生	zzz	2

【多对多】

建表原则：创建一个中间表，中间表中至少两个字段作为外键分别指向多对多双方的主键。

学生表：

SID	SNAME
1	张三
2	李四

课程表：

CID	CNAME
001	PHP
002	Java
003	C++

学生选课表

SNO	CNO
1	001
1	002
2	002

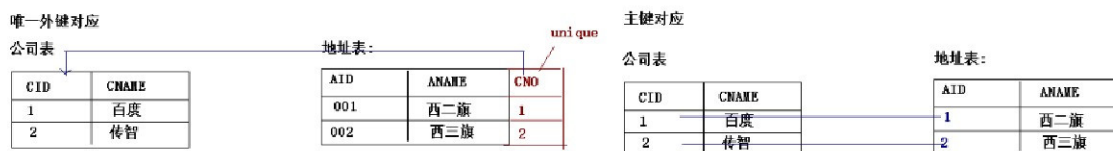
一对一

【一对一】

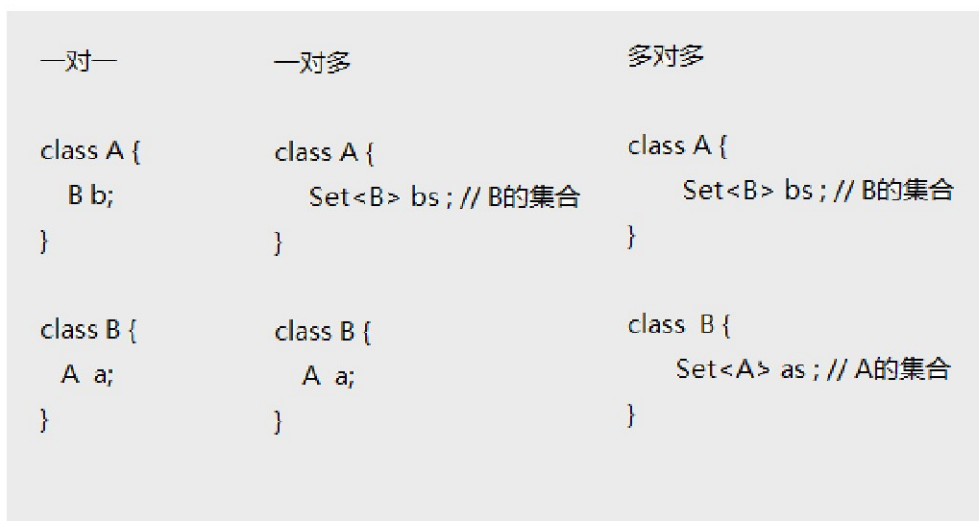
建表原则有两种：

一种：唯一外键对应：假设一对一种的任意一方为多，在多的的一方创建外键指向一的一方的主键，然后将外键设置为唯一。

二种：主键对应：一方的主键作为另一方的主键。



数据库表能够描述的实体数据之间的关系，通过对象也可以进行描述，所谓的关联映射就是将关联关系映射到数据库里，在对象模型中就是一个或多个引用。在 Hibernate 中采用 Java 对象关系来描述数据表之间的关系，具体如图所示。



从图可以看出，通过一对一的关系就是在本类中定义对方类型的对象，如 A 中定义 B 类类型的属性 b，B 类中定义 A 类类型的属性 a；一对多的关系，图中描述的是一个 A 对应多个 B 类类型的情况，需要在 A 类以 Set 集合的方式引入 B 类型的对象，在 B 类中定义 A 类类型的属性 a；多对多的关系，在 A 类中定义 B 类类型的 Set 集合，在 B 类中定义 A 类类型的 Set 集合，这里用 Set 集合的目的是避免了数据的重复。

以上就是系统模型中实体设计的三种关联关系，由于一对一的关联关系在开发中不常使用，所以我们不单独讲解，了解即可。那么接下来我们就先来学习下一对多的关系映射吧。

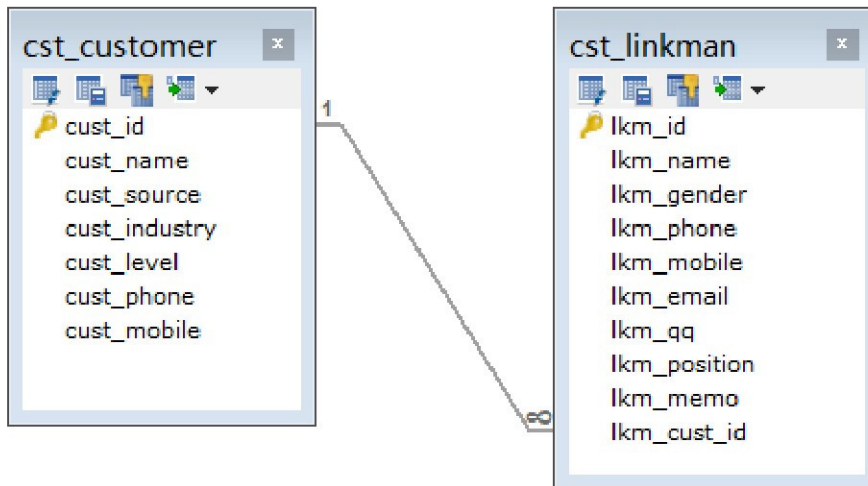
1.2.2 Hibernate 的一对多关联映射

1.2.2.1 创建表：

创建联系人表 cst_linkman。

导入 crm/sql/crm_cst_linkman.sql

联系人表中存在外键 (lkm_cust_id 即所属客户 id)，外键指向客户表：



```
ALTER TABLE cst_linkman
ADD CONSTRAINT `FK_cst_linkman_lkm_cust_id` FOREIGN KEY (`lkm_cust_id`) REFERENCES
`cst_customer` (`cust_id`) ON DELETE NO ACTION ON UPDATE NO ACTION
```

1.2.2.2 创建实体:

【客户的实体】:

```
public class Customer {
    private Long cust_id;
    private String cust_name;
    private String cust_source;
    private String cust_industry;
    private String cust_level;
    private String cust_phone;
    private String cust_mobile;
    // 一个客户有多个联系人: 客户中应该放有联系人的集合.
    private Set<LinkMan> linkMans = new HashSet<LinkMan>();

    public Long getCust_id() {
        return cust_id;
    }
    public void setCust_id(Long cust_id) {
        this.cust_id = cust_id;
    }
    public String getCust_name() {
        return cust_name;
    }
    public void setCust_name(String cust_name) {
        this.cust_name = cust_name;
    }
    public String getCust_source() {
```




```
        return cust_source;
    }

    public void setCust_source(String cust_source) {
        this.cust_source = cust_source;
    }

    public String getCust_industry() {
        return cust_industry;
    }

    public void setCust_industry(String cust_industry) {
        this.cust_industry = cust_industry;
    }

    public String getCust_level() {
        return cust_level;
    }

    public void setCust_level(String cust_level) {
        this.cust_level = cust_level;
    }

    public String getCust_phone() {
        return cust_phone;
    }

    public void setCust_phone(String cust_phone) {
        this.cust_phone = cust_phone;
    }

    public String getCust_mobile() {
        return cust_mobile;
    }

    public void setCust_mobile(String cust_mobile) {
        this.cust_mobile = cust_mobile;
    }

    public Set<LinkMan> getLinkMans() {
        return linkMans;
    }

    public void setLinkMans(Set<LinkMan> linkMans) {
        this.linkMans = linkMans;
    }
}
```

【联系人的实体】：

```
public class LinkMan {
    private Long lkm_id;
    private String lkm_name;
    private String lkm_gender;
    private String lkm_phone;
    private String lkm_mobile;
```



```
private String lkm_email;
private String lkm_qq;
private String lkm_position;
private String lkm_memo;

// Hibernate是一个 ORM 的框架:在关系型的数据库中描述表与表之间的关系,使用的是外键.开发语言使用是 Java, 面向对象的.

private Customer customer;

public Long getLkm_id() {
    return lkm_id;
}

public void setLkm_id(Long lkm_id) {
    this.lkm_id = lkm_id;
}

public String getLkm_name() {
    return lkm_name;
}

public void setLkm_name(String lkm_name) {
    this.lkm_name = lkm_name;
}

public String getLkm_gender() {
    return lkm_gender;
}

public void setLkm_gender(String lkm_gender) {
    this.lkm_gender = lkm_gender;
}

public String getLkm_phone() {
    return lkm_phone;
}

public void setLkm_phone(String lkm_phone) {
    this.lkm_phone = lkm_phone;
}

public String getLkm_mobile() {
    return lkm_mobile;
}

public void setLkm_mobile(String lkm_mobile) {
    this.lkm_mobile = lkm_mobile;
}

public String getLkm_email() {
    return lkm_email;
}

public void setLkm_email(String lkm_email) {
    this.lkm_email = lkm_email;
}
```



```
public String getLkm_qq() {  
    return lkm_qq;  
}  
  
public void setLkm_qq(String lkm_qq) {  
    this.lkm_qq = lkm_qq;  
}  
  
public String getLkm_position() {  
    return lkm_position;  
}  
  
public void setLkm_position(String lkm_position) {  
    this.lkm_position = lkm_position;  
}  
  
public String getLkm_memo() {  
    return lkm_memo;  
}  
  
public void setLkm_memo(String lkm_memo) {  
    this.lkm_memo = lkm_memo;  
}  
  
public Customer getCustomer() {  
    return customer;  
}  
  
public void setCustomer(Customer customer) {  
    this.customer = customer;  
}  
  
}
```

1.2.2.3 创建映射:

【客户的映射】

```
<hibernate-mapping>  
    <class name="cn.itcast.hibernate.domain.Customer" table="cst_customer">  
        <id name="cust_id">  
            <generator class="native"/>  
        </id>  
  
        <property name="cust_name" length="32"/>  
        <property name="cust_source" column="cust_source"/>  
        <property name="cust_industry" column="cust_industry"/>  
        <property name="cust_level" column="cust_level"/>  
        <property name="cust_phone" column="cust_phone"/>  
        <property name="cust_mobile" column="cust_mobile"/>  
    </class>  
</hibernate-mapping>
```



```

<!-- 配置关联对象 -->
<!--
    set 标签:
        * name 属性:多的一方的集合的属性名称.
-->
<set name="linkMans">
    <!--
        key 标签 :
            * column 属性:多的一方的外键的名称.
-->
    <key column="lkm_cust_id"></key>
    <!--
        one-to-many 标签:
            * class 属性:多的一方的类全路径
-->
    <one-to-many class="cn.itcast.hibernate.domain.LinkMan"/>
</set>
</class>
</hibernate-mapping>

```

使用 set 集合来描述 Customer.java 类中的属性 linkMans。在 Hibernate 的映射文件中，使用<set> 标签用来描述被映射类中的 Set 集合，<key>标签的 column 属性值对应文件多的一方的外键名称，在 Customer.java 客户类中，客户与联系人是一对多的关系，Hibernate 的映射文件中，使用<one-to-many>标签来描述持久化类的一对多关联，其中 class 属性用来描述映射的关联类。

【联系人映射】

```

<hibernate-mapping>
    <class name="cn.itcast.hibernate.domain.LinkMan" table="cst_linkman">
        <id name="lkm_id" column="lkm_id">
            <generator class="native"/>
        </id>

        <property name="lkm_name"/>
        <property name="lkm_gender"/>
        <property name="lkm_phone"/>
        <property name="lkm_mobile"/>
        <property name="lkm_email"/>
        <property name="lkm_qq"/>
        <property name="lkm_position"/>
        <property name="lkm_memo"/>

        <!-- 配置关联对象: -->
        <!--
            many-to-one: 标签. 代表多对一.
            * name          : 一的一方的对象的名称.

```




```

        * class      :一的一方的类的全路径。
        * column :表中的外键的名称。

-->
<many-to-one name="customer" class="cn.itcast.hibernate.domain.Customer"
column="lkm_cust_id"/>
</class>
</hibernate-mapping>

```

<many-to-one>标签定义两个持久化类的关联，这种关联是数据表间的多对一关联，联系人与客户就是多对一的关系，所以用<many-to-one>标签来描述。<many-to-one>标签的 name 属性用来描述 customer 在 LinkMan.java 类中的属性的名称，class 属性用来指定映射的类，column 属性值对应表中的外键列名。

1.2.2.4 将映射添加到配置文件

```

<mapping resource="com/itheima/domain/Customer.hbm.xml"/>
<mapping resource="com/itheima/domain/LinkMan.hbm.xml"/>

```

1.2.2.5 编写测试代码：

```

@Test
// 保存一个客户和两个联系人：
public void demol() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();
    // 创建一个客户
    Customer customer = new Customer();
    customer.setCust_name("姜总");

    // 创建两个联系人：
    LinkMan linkMan1 = new LinkMan();
    linkMan1.setLkm_name("李秘书");
    LinkMan linkMan2 = new LinkMan();
    linkMan2.setLkm_name("王助理");

    // 建立关系：
    customer.getLinkMans().add(linkMan1);
    customer.getLinkMans().add(linkMan2);
    linkMan1.setCustomer(customer);
    linkMan2.setCustomer(customer);

    session.save(customer);
    session.save(linkMan1);
}

```



```
session.save(linkMan2);

transaction.commit();
}
```

在配置文件中添加了自动建表信息后，运行程序时，程序会自动创建两张表，并且插入数据。使用 Junit4 运行 demo1()方法后，控制台输出结果，如图所示

```
Hibernate:
insert
into
  cst_customer
(cust_name, cust_source, cust_industry, cust_level, cust_phone, cust_mobile)
values
  (?, ?, ?, ?, ?, ?)
Hibernate:
insert
into
  cst_linkman
(lkm_name, lkm_gender, lkm_phone, lkm_mobile, lkm_email, lkm_qq, lkm_position, lkm_memo, lkm_cust_id)
values
  (?, ?, ?, ?, ?, ?, ?, ?, ?)
Hibernate:
insert
into
  cst_linkman
(lkm_name, lkm_gender, lkm_phone, lkm_mobile, lkm_email, lkm_qq, lkm_position, lkm_memo, lkm_cust_id)
values
  (?, ?, ?, ?, ?, ?, ?, ?, ?)
Hibernate:
update
  cst_linkman
set
  lkm_cust_id=?
where
  lkm_id=?
Hibernate:
update
  cst_linkman
set
  lkm_cust_id=?
where
  lkm_id=?
```

从图的输出结果可以看到，控制台成功输出了三条 insert 语句和两条 update 语句，此时查询数据库两张表及表中的数据后，查询结果如图所示。

1 结果

2 分析

3 信息

4 表数据

5 资料

6 历史

</

1 结果	2 分析	3 信息	4 表数据	5 资料	6 历史
</					

从上图的查询结果可以看出，数据表创建成功，并成功插入了相应数据。那么一个基本的一对多的关联关系映射就已经配置好了。以上我们的代码可以发现我们建立的关系是双向的，即客户关联了联系人，同时联系人也关联了客户。

```
//客户关联了联系人
customer.getLinkMans().add(linkMan1);
customer.getLinkMans().add(linkMan2);
//联系人关联客户
linkMan1.setCustomer(customer);
linkMan2.setCustomer(customer);
```

这就是双向关联，那么既然已经进行了双向的关联关系的设置，那么我们还保存了双方，那如



果我们只保存一方是否可以呢？也就是说我们建立了双向的维护关系，只保存客户或者只保存联系人是否可以。那么我们来进行一次测试。

```
@Test
// 保存操作只保存一个方向是否可以？
public void demo2() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();
    // 创建一个客户
    Customer customer = new Customer();
    customer.setCust_name("刘总");

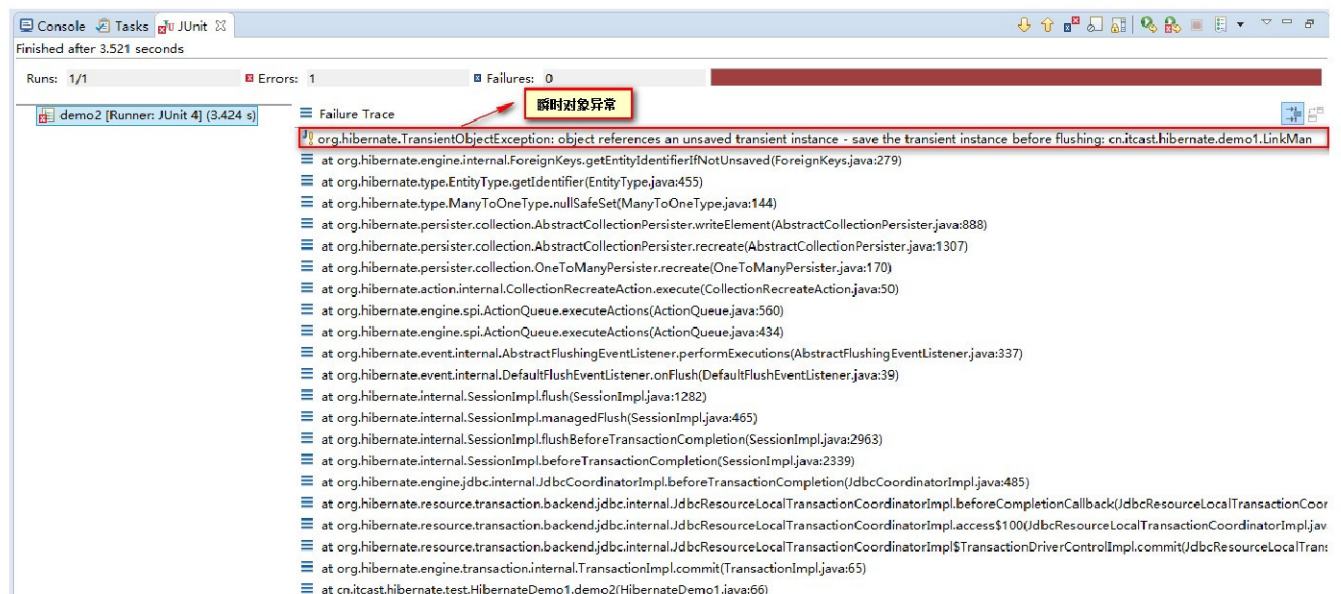
    // 创建两个联系人：
    LinkMan linkMan1 = new LinkMan();
    linkMan1.setLkm_name("王秘书");

    // 建立关系：
    customer.getLinkMans().add(linkMan1);
    linkMan1.setCustomer(customer);

    session.save(customer); // 瞬时对象异常，持久态的对象关联了一个瞬时态对象的异常。
    // session.save(linkMan1);

    transaction.commit();
}
```

我们执行这段代码，会出现如下错误：



这样操作显然不行，无论从那一方保存都会出现同样的异常：瞬时对象异常，一个持久态对象关联了一个瞬时态对象，那就说明我们不能只保存一方。那如果我们就想只保存一个方向应该如何进行操作呢，那么我们可以使用 Hibernate 的级联操作，那么我们来看下 Hibernate 的级联吧。



1.2.3 一对多的相关操作:

级联操作是指当主控方执行保存、更新或者删除操作时，其关联对象（被控方）也执行相同的操作。在映射文件中通过对 cascade 属性的设置来控制是否对关联对象采用级联操作，级联操作对各种关联关系都是有效的。

1.2.3.1 级联保存或更新

级联是有方向性的，所谓的方向性指的是，在保存一的一方级联多的一方和在保存多的一方级联一的一方。

【保存客户级联联系人】

首先要确定我们要保存的主控方是那一方，我们要保存客户，所以客户是主控方，那么需要在客户的映射文件中进行如下的配置。

```
<!-- 配置关联关系映射: -->
<!-- 配置一个set标签:代表一个集合 name属性: 集合的属性名称 -->
<set name="LinkMans" cascade="save-update">
    <!-- 配置一个key标签 column属性:多的一方的外键的名称 -->
    <key column="lkm_cust_id"/>
    <!-- 配置one-to-many class对用多的一方的类的全路径 -->
    <one-to-many class="cn.itcast.hibernate.demo1.LinkMan"/>
</set>
```

然后我们就可以编写测试代码了，代码如下：

```
@Test
// 级联保存：只保存一边的问题。
// 级联是有方向性，保存客户同时级联客户的联系人。
// 在 Customer.hbm.xml 中的<set>标签上配置 cascade="save-update"
public void demo3(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();
    // 创建一个客户
    Customer customer = new Customer();
    customer.setCust_name("刘总");

    // 创建两个联系人：
    LinkMan linkMan1 = new LinkMan();
    linkMan1.setLkm_name("王秘书");

    // 建立关系：
    customer.getLinkMans().add(linkMan1);
    linkMan1.setCustomer(customer);
}
```



```
session.save(customer);

transaction.commit();
}
```

【保存联系人级联客户】

同样我们需要确定主控方，现在我们的主控方是联系人。所以需要在联系人的映射文件中进行配置，内容如下：

```
<!-- 关联关系的映射的配置：联系人关联客户 -->
<!-- 在多方配置 many-to-one：代表多对一 -->
<!--
    many-to-one标签：代表多对一
    * name           : 一的一方的对象的属性名称。
    * class           : 一的一方的类的全路径
    * column          : 在多方的一方的外键的名称。
-->
<many-to-one name="customer" cascade="save-update" class="cn.itcast.hibernate.demo1.Customer" column="lkm_cust_id"/>
```

然后我们就可以编写代码进行测试了。测试代码如下：

```
@Test
// 级联保存：只保存一边的问题。
// 级联是有方向性，保存联系人同时级联客户。
// 在 LinkMan.hbm.xml 中在<many-to-one>标签上配置 cascade="save-update"
public void demo4() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();

    // 创建一个客户
    Customer customer = new Customer();
    customer.setCust_name("刘总");

    // 创建两个联系人：
    LinkMan linkMan1 = new LinkMan();
    linkMan1.setLkm_name("王秘书");

    // 建立关系：
    customer.getLinkMans().add(linkMan1);
    linkMan1.setCustomer(customer);

    session.save(linkMan1);

    transaction.commit();
}
```

到这我们已经可以看到级联保存或更新的效果了。那么我们维护的时候都是双向的关系维护。那么这种关系到底表述的是什么含义呢？我们可以通过下面的测试来更进一步了解关系的维护（对象导航）的含义。



1.2.3.2 测试对象的导航的问题

我们所说的对象导航其实就是在维护双方的关系。

```
customer.getLinkMans().add(linkMan1);  
linkMan1.setCustomer(customer)
```

这种关系有什么用途呢？我们可以通过下面的测试来更进一步学习 Hibernate。

我们在客户和联系人端都配置了 `cascade="save-update"`，然后进行如下的关系设置。会产生什么样的效果呢？

```
1      @Test  
2      // 测试对象导航和级联操作：  
3      public void demo5(){  
4          Session session = HibernateUtils.getCurrentSession();  
5          Transaction transaction = session.beginTransaction();  
6          // 创建一个客户  
7          Customer customer = new Customer();  
8          customer.setCust_name("刘总");  
9  
10         // 创建三个联系人：  
11         LinkMan linkMan1 = new LinkMan();  
12         linkMan1.setLkm_name("王秘书");  
13         LinkMan linkMan2 = new LinkMan();  
14         linkMan2.setLkm_name("李秘书");  
15         LinkMan linkMan3 = new LinkMan();  
16         linkMan3.setLkm_name("张助理");  
17  
18         // 建立关系：  
19         linkMan1.setCustomer(customer);  
20         customer.getLinkMans().add(linkMan2);  
21         customer.getLinkMans().add(linkMan3);  
22  
23         // 条件是双方都配置了 cascade="save-update"  
24         session.save(linkMan1); // 数据库中有几条记录？发送几条 insert 语句。4 条  
25         // session.save(customer); // 发送几条 insert 语句？3 条  
26         // session.save(linkMan2); // 发送几条 insert 语句？1 条  
27         transaction.commit();  
28     }
```

我们执行第 24 行的时候，问会执行几条 insert 语句呢？其实发现会有 4 条 insert 语句的，因为联系人 1 关联了客户，客户又关联了联系人 2 和联系人 3，所以当保存联系人 1 的时候，联系人 1 是可以进入到数据库的，它关联的客户也是会进入到数据库的(因为联系人一端配置了级联)，那么客户进入到数据库以后，联系人 2 和联系人 3 也同样会进入到数据库(因为客户一端配置了级联)，所以会有 4 条 insert 语句。

我们执行 25 行的时候，问会有几条 insert 语句？这时我们保存的客户对象，可以对象关联了联系人 2 和联系人 3，那么客户在保存进入数据库的时候，联系人 2 和联系人 3 也会进入到数据库，



所以是 3 条 insert 语句。

同理我们执行 26 行代码的时候，只会执行 1 条 insert 语句，因为联系人 2 会保存到数据库，但是联系人 2 没有客户客户建立关系。所以客户不会保存到数据库。

到这我们应该更加理解了 Hibernate 的这种关系的建立和级联的关系了。那么级联还有哪些操作呢？接下来我们来学习级联删除的操作。

1.2.3.3 Hibernate 的级联删除

我们之前学习过级联保存或更新，那么再来看级联删除也就不难理解了，级联删除也是有方向性的，删除客户同时级联删除联系人，也可以删除联系人同时级联删除客户（这种需求很少）。

原来 JDBC 中删除客户和联系人的时候，如果有外键的关系是不可以删除的，但是现在我们使用了 Hibernate，其实 Hibernate 可以实现这样的功能，但是不会删除客户同时删除联系人，默认情况下 Hibernate 会怎么做呢？我们来看下面的测试：

```
@Test
// 删除有关联关系的对象：(默认情况：先将关联对象的外键置为 NULL 删除客户对象)
public void demo6() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();

    Customer customer = session.get(Customer.class, 11);
    session.delete(customer);

    transaction.commit();
}
```

默认的情况下如果客户下面还有联系人，Hibernate 会将联系人的外键置为 null，然后去删除客户。那么其实有的时候我们需要删除客户的时候，同时将客户关联的联系人一并删除。这个时候我们就需要使用 Hibernate 的级联保存操作了。

【删除客户的时候同时删除客户的联系人】

确定删除的主控方式客户，所以需要在客户端配置：

```
<!-- 配置关联关系映射：-->
<!-- 配置一个set标签：代表一个集合 name属性：集合的属性名称 -->
<set name="LinkMans" cascade="delete">
    <!-- 配置一个key标签 column属性：多的一方的外键的名称 -->
    <key column="lkm_cust_id"/>
    <!-- 配置one-to-many class对用多的一方的类的全路径 -->
    <one-to-many class="cn.itcast.hibernate.demo1.LinkMan"/>
</set>
```

如果还想有之前的级联保存或更新，同时还想有级联删除，那么我们可以进行如下的配置：



```
<!-- 配置关联关系映射: -->
<!-- 配置一个set标签:代表一个集合 name属性: 集合的属性名称 -->
<set name="LinkMans" cascade="delete,save-update">
    <!-- 配置一个key标签 column属性:多的一方的外键的名称 -->
    <key column="lkm_cust_id"/>
    <!-- 配置one-to-many class对用多的一方的类的全路径 -->
    <one-to-many class="cn.itcast.hibernate.demo1.LinkMan"/>
</set>
```

配置完成以后我们可以来编写测试代码了，代码如下：

```
@Test
// 级联删除:级联删除有方向性.
// 删除客户同时级联删除联系人.
// 在 Customer.hbm.xml 中 set 标签上配置 cascade="delete"
public void demo7() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();

    // 级联删除: 必须是先查询在删除的.
    // 因为查询到客户, 这个时候客户的联系人的集合中就会有数据.
    Customer customer = session.get(Customer.class, 11);
    session.delete(customer);

    transaction.commit();
}
```

【删除联系人的时候同时删除客户】

同样我们删除的是联系人，那么联系人是主控方，需要在联系人端配置：

```
<!-- 关联关系的映射的配置: 联系人关联客户 -->
<!-- 在多方配置 many-to-one:代表多对一 -->
<!--
    many-to-one标签:代表多对一
    * name      :一的一方的对象的属性名称.
    * class     :一的一方的类的全路径
    * column    :在多方的一方的外键的名称.
-->
<many-to-one name="customer" cascade="delete" class="cn.itcast.hibernate.demo1.Customer" column="lkm_cust_id"/>
```

如果需要既做保存或更新有有级联删除的功能，也可以如下配置：

```
<!-- 关联关系的映射的配置: 联系人关联客户 -->
<!-- 在多方配置 many-to-one:代表多对一 -->
<!--
    many-to-one标签:代表多对一
    * name      :一的一方的对象的属性名称.
    * class     :一的一方的类的全路径
    * column    :在多方的一方的外键的名称.
-->
<many-to-one name="customer" cascade="delete,save-update" class="cn.itcast.hibernate.demo1.Customer" column="lkm_cust_id"/>
```

编写测试代码如下：

```
@Test
// 级联删除:级联删除有方向性.
// 删除联系人同时级联删除客户.
// 在 LinkMan.hbm.xml 中 many-to-one 标签上配置 cascade="delete"
public void demo8() {
    Session session = HibernateUtils.getCurrentSession();
```




```
Transaction transaction = session.beginTransaction();

// 级联删除：必须是先查询在删除的
LinkMan linkMan = session.get(LinkMan.class, 31);
session.delete(linkMan);

transaction.commit();
}
```

1.2.3.4 双向关联产生多余的 SQL 语句

到这我们已经了解了 Hibernate 级联的基本配置和使用。但是有些时候我们需要进行如下的操作：
数据库中记录如下：

客户表

1 结果

2 分析

3 信息

4 表数据

5 资料

6 历史

联系人表:

1 结果										2 分析										3 信息										4 表数据										5 资料										6 历史																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																									
☐ 所有行										☒ 行的范围										首行: 0										100 行										刷新																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																			
										lkm_id										lkm_name										lkm_gender										lkm_phone										lkm_mobile										lkm_email										lkm_qq										lkm_position										lkm_memo										lkm_cust_id																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																							
☐										1										王助理										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)										(NULL)									

需要将 2 号联系人关联给 2 号客户。也就是将 2 号李秘书这个联系人关联给 2 号刘总这个客户。
编写修改 2 号客户关联的联系人代码。

```
@Test
// 将 2 号联系人关联的客户改为 2 号客户。
public void demo9() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();

    Customer customer = session.get(Customer.class, 21);
    LinkMan linkMan = session.get(LinkMan.class, 21);

    linkMan.setCustomer(customer);
    customer.getLinkMans().add(linkMan);

    transaction.commit();
}
```




运行该代码，控制台会输出如下内容：

```
Hibernate:
  select
    customer0_.cust_id as cust_id1_0_,
    customer0_.cust_name as cust_name2_0_,
    customer0_.cust_source as cust_sou3_0_,
    customer0_.cust_industry as cust_ind4_0_,
    customer0_.cust_level as cust_level5_0_,
    customer0_.cust_phone as cust_pho6_0_,
    customer0_.cust_mobile as cust_mob7_0_
  from
    cst_customer customer0_
  where
    customer0_.cust_id=?
Hibernate:
  select
    linkman0_.lkm_id as lkm_id1_1_0_,
    linkman0_.lkm_name as lkm_name2_1_0_,
    linkman0_.lkm_gender as lkm_gender3_1_0_,
    linkman0_.lkm_phone as lkm_phon4_1_0_,
    linkman0_.lkm_mobile as lkm_mobi5_1_0_,
    linkman0_.lkm_email as lkm_email6_1_0_,
    linkman0_.lkm_qq as lkm_qq7_1_0_,
    linkman0_.lkm_position as lkm_posi8_1_0_,
    linkman0_.lkm_memo as lkm_memo9_1_0_,
    linkman0_.lkm_cust_id as lkm_cust10_1_0_
  from
    cst_linkman linkman0_
  where
    linkman0_.lkm_id=?
Hibernate:
  select
    linkmans0_.lkm_cust_id as lkm_cust10_1_1_0_,
    linkmans0_.lkm_id as lkm_id1_1_1_0_,
    linkmans0_.lkm_id as lkm_id1_1_1_1_,
    linkmans0_.lkm_name as lkm_name2_1_1_1_,
    linkmans0_.lkm_gender as lkm_gender3_1_1_1_,
    linkmans0_.lkm_phone as lkm_phon4_1_1_1_,
    linkmans0_.lkm_mobile as lkm_mobi5_1_1_1_,
    linkmans0_.lkm_email as lkm_email6_1_1_1_,
    linkmans0_.lkm_qq as lkm_qq7_1_1_1_,
    linkmans0_.lkm_position as lkm_posi8_1_1_1_,
    linkmans0_.lkm_memo as lkm_memo9_1_1_1_,
    linkmans0_.lkm_cust_id as lkm_cust10_1_1_1_
  from
    cst_linkman linkmans0_
  where
    linkmans0_.lkm_cust_id=?
Hibernate:
  update
    cst_linkman
  set
    lkm_name=?,
    lkm_gender=?,
    lkm_phone=?,
    lkm_mobile=?,
    lkm_email=?,
    lkm_qq=?,
    lkm_position=?,
    lkm_memo=?,
    lkm_cust_id=?
  where
    lkm_id=?
Hibernate:
  update
    cst_linkman
  set
    lkm_cust_id=?
  where
    lkm_id=?
```

我们会发现执行了两次 update 语句，其实这两个 update 都是修改了外键的操作，那么为什么发送两次呢？那么我们来分析一下产生这种问题的原因：



双方维护关系:产生多余的SQL.

```
@Test
// 将2号联系人关联的客户改为2号客户。
public void demo10(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();

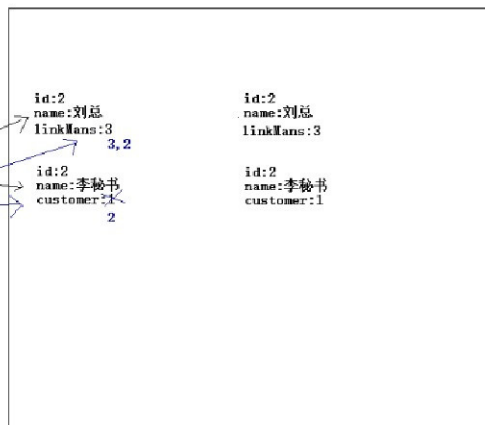
    Customer customer = session.get(Customer.class, 21);
    LinkMan linkMan = session.get(LinkMan.class, 21);

    linkMan.setCustomer(customer);
    customer.getLinkMans().add(linkMan);

    transaction.commit();
}
```

缓冲区和快照区的数据不一致自动更新数据库
发送两条更新操作，更新外键了。

一级缓存区



我们已经分析过了，因为双向维护了关系，而且持久态对象可以自动更新数据库，更新客户的时候会修改一次外键，更新联系人的时候同样也会修改一次外键。这样就会产生多余的 SQL，那么问题产生了，我们又该如何解决呢？

其实解决的办法很简单，只需要将一方放弃外键维护权即可。也就是说关系不是双方维护的，只需要交给某一方去维护就可以了。通常我们都是交给多的一方去维护的。为什么呢？因为多的一方才是维护关系的最好的地方，举个例子，一个老师对应多个学生，一个学生对应一个老师，这是典型的一对多。那么一个老师如果要记住所有学生的名字很难的，但如果让每个学生记住老师的名字应该不难。其实就是这个道理。所以在一对多中，一的一方都会放弃外键的维护权（关系的维护）。

这个时候如果能让一的一方放弃外键的维护权，只需要进行如下的配置即可。

```
<!-- 配置关联关系映射: -->
<!-- 配置一个set标签:代表一个集合 name属性: 集合的属性名称 -->
<set name="linkMans" cascade="delete,save-update" inverse="true">
    <!-- 配置一个key标签 column属性:多的一方的外键的名称 -->
    <key column="lkm_cust_id"/>
    <!-- 配置one-to-many class对用多的一方的类的全路径 -->
    <one-to-many class="cn.itcast.hibernate.demo1.LinkMan"/>
</set>
```

inverse 的默认值是 false，代表不放弃外键维护权，配置值为 true，代表放弃了外键的维护权。这个时候再来执行之前的操作：



```

Hibernate:
select
    customer0_.cust_id as cust_id1_0_,
    customer0_.cust_name as cust_name2_0_,
    customer0_.cust_source as cust_sou3_0_,
    customer0_.cust_industry as cust_ind4_0_,
    customer0_.cust_level as cust_level5_0_,
    customer0_.cust_phone as cust_pho6_0_,
    customer0_.cust_mobile as cust_mob7_0_
from
    cst_customer customer0_
where
    customer0_.cust_id=?
Hibernate:
select
    linkman0_.lkm_id as lkm_id1_1_0_,
    linkman0_.lkm_name as lkm_name2_1_0_,
    linkman0_.lkm_gender as lkm_gender3_1_0_,
    linkman0_.lkm_phone as lkm_phon4_1_0_,
    linkman0_.lkm_mobile as lkm_mobi5_1_0_,
    linkman0_.lkm_email as lkm_email6_1_0_,
    linkman0_.lkm_qq as lkm_qq7_1_0_,
    linkman0_.lkm_position as lkm_posi8_1_0_,
    linkman0_.lkm_memo as lkm_memo9_1_0_,
    linkman0_.lkm_cust_id as lkm_cust10_1_0_
from
    cst_linkman linkman0_
where
    linkman0_.lkm_id=?
Hibernate:
select
    linkmans0_.lkm_cust_id as lkm_cust10_1_1_0_,
    linkmans0_.lkm_id as lkm_id1_1_0_,
    linkmans0_.lkm_id as lkm_id1_1_1_0_,
    linkmans0_.lkm_name as lkm_name2_1_1_0_,
    linkmans0_.lkm_gender as lkm_gender3_1_1_0_,
    linkmans0_.lkm_phone as lkm_phon4_1_1_0_,
    linkmans0_.lkm_mobile as lkm_mobi5_1_1_0_,
    linkmans0_.lkm_email as lkm_email6_1_1_0_,
    linkmans0_.lkm_qq as lkm_qq7_1_1_0_,
    linkmans0_.lkm_position as lkm_posi8_1_1_0_,
    linkmans0_.lkm_memo as lkm_memo9_1_1_0_,
    linkmans0_.lkm_cust_id as lkm_cust10_1_1_0_
from
    cst_linkman linkmans0_
where
    linkmans0_.lkm_cust_id=?
Hibernate:
update
    cst_linkman
set
    lkm_name=?,
    lkm_gender=?,
    lkm_phone=?,
    lkm_mobile=?,
    lkm_email=?,
    lkm_qq=?,
    lkm_position=?,
    lkm_memo=?,
    lkm_cust_id=?
where
    lkm_id=?

```

这个时候我们会发现就不会出现上述的问题了，不会产生多余的 SQL 了（因为一的一方已经放弃了外键的维护权）。

那么这个问题我们已经解决了，但是有很多同学对应 cascade 和 inverse 还是不是太懂，我们可以通过下面的案例区分 cascade 和 inverse 的区别，因为这两个参数我们以后的开发中会经常使用。

1.2.3.5 区分 cascade 和 inverse:

```

@Test
// cascade 和 inverse
// cascade 强调的是操作一个对象的时候，是否操作其关联对象
// inverse 强调的是外键的维护权。
public void demo10() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();

```



```
Customer customer = new Customer();
customer.setCust_name("王总");

LinkMan linkMan = new LinkMan();
linkMan.setLkm_name("李秘书");
// 在 Customer.hbm.xml 中的 set 上配置 cascade="save-update" inverse="true"
customer.getLinkMans().add(linkMan);

session.save(customer); // 客户关联的联系人是否被保存到数据库, 外键是啥?

transaction.commit();
}
```

这个时候我们会发现，如果在 set 集合上配置 cascade="save-update" inverse="true" 了，那么执行保存客户的操作，会发现客户和联系人都进入到数据库了，但是没有外键，是因为配置了 cascade 了所以客户关联的联系人会进入到数据库，但是客户一端放弃了外键维护权，所以联系人插入到数据库以后是没有外键的。

一对多已经完成了，那么我们来看下 Hibernate 中的多对多的关系的配置。

案例二：完成多对多的关联关系映射并操作

1.3 案例需求

1.3.1 需求描述

用户即使用系统的用户，包括业务员、总经理等角色，不同类型的用户使用系统不同的功能，本功能要完成给用户分配角色，功能包括：给用户分配权限、取消用户分配的角色。

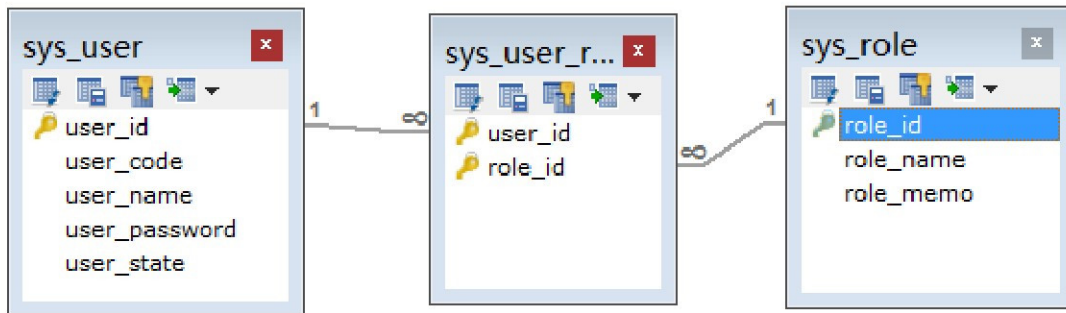
1.4 相关知识点

1.4.1 Hibernate 的多对多关联关系映射

1.4.1.1 创建表：

创建 sys_user、sys_role、sys_user_role 三张表。

数据模型如下：



先后导入下边的 sql:
crm_sys_user.sql
crm_sys_role.sql
crm_sys_user_role.sql

1.4.1.2 创建实体

【用户实体】

```
public class User {
    private Long user_id;
    private String user_code;
    private String user_name;
    private String user_password;
    private String user_state;
    // 用户所属的角色的集合:
    private Set<Role> roles = new HashSet<Role>();
    public Long getUser_id() {
        return user_id;
    }
    public void setUser_id(Long user_id) {
        this.user_id = user_id;
    }
    public String getUser_code() {
        return user_code;
    }
    public void setUser_code(String user_code) {
        this.user_code = user_code;
    }
    public String getUser_name() {
        return user_name;
    }
    public void setUser_name(String user_name) {
        this.user_name = user_name;
    }
    public String getUser_password() {
```



```
        return user_password;
    }

    public void setUser_password(String user_password) {
        this.user_password = user_password;
    }

    public String getUser_state() {
        return user_state;
    }

    public void setUser_state(String user_state) {
        this.user_state = user_state;
    }

    public Set<Role> getRoles() {
        return roles;
    }

    public void setRoles(Set<Role> roles) {
        this.roles = roles;
    }
}
```

【角色实体】

```
public class Role {
    private Long role_id;
    private String role_name;
    private String role_memo;
    // 一个角色包含多个用户:
    private Set<User> users = new HashSet<User>();
    public Long getRole_id() {
        return role_id;
    }

    public void setRole_id(Long role_id) {
        this.role_id = role_id;
    }

    public String getRole_name() {
        return role_name;
    }

    public void setRole_name(String role_name) {
        this.role_name = role_name;
    }

    public String getRole_memo() {
        return role_memo;
    }

    public void setRole_memo(String role_memo) {
        this.role_memo = role_memo;
    }
}
```



```
public Set<User> getUsers() {  
    return users;  
}  
  
public void setUsers(Set<User> users) {  
    this.users = users;  
}  
  
}
```

1.4.1.3 创建映射

【用户的映射:】

```
<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE hibernate-mapping PUBLIC  
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"  
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">  
<hibernate-mapping>  
    <class name="cn.itcast.hibernate.demo2.User" table="sys_user">  
        <id name="user_id">  
            <generator class="native"/>  
        </id>  
  
        <property name="user_code"/>  
        <property name="user_name"/>  
        <property name="user_password"/>  
        <property name="user_state"/>  
  
        <!-- 关联关系的映射的配置 -->  
        <!--  
            set 标签:  
                name      :关联的另一方的集合的属性名称.  
                table     :中间表的名称  
        -->  
        <set name="roles" table="sys_user_role">  
            <!--  
                key 标签:  
                    column :当前对象在中间表中的外键的名称.  
            -->  
            <key column="user_id"/>  
            <!--  
                many-to-many 标签  
                    class      :关联另一方的类的全路径.  
                    column     :关联的另一方在中间表的外键名称
```



```

-->
        <many-to-many                                class="cn.itcast.hibernate.demo2.Role"
column="role_id"/>
        </set>
    </class>
</hibernate-mapping>

```

【角色的映射:】

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="cn.itcast.hibernate.demo2.Role" table="sys_role">
        <id name="role_id">
            <generator class="native"/>
        </id>

        <property name="role_name"/>
        <property name="role_memo"/>

        <set name="users" table="sys_user_role">
            <key column="role_id"/>
            <many-to-many                                class="cn.itcast.hibernate.demo2.User"
column="user_id"/>
        </set>
    </class>
</hibernate-mapping>

```

1.4.1.4 在核心配置中加入映射文件

```

<!-- 加载映射 -->
<mapping resource="com/itheima/domain/User.hbm.xml"/>
<mapping resource="com/itheima/domain/Role.hbm.xml"/>

```

1.4.1.5 编写测试类

```

@Test
// 保存用户和角色
public void demo1() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction transaction = session.beginTransaction();
}

```




```
User user1 = new User();
user1.setUser_name("张三");

User user2 = new User();
user2.setUser_name("李四");

Role role1 = new Role();
role1.setRole_name("前台");
Role role2 = new Role();
role2.setRole_name("人事");
Role role3 = new Role();
role3.setRole_name("助理");

// 如果多对多建立了双向关联，一定要有一方放弃外键维护权。
user1.getRoles().add(role1);
user1.getRoles().add(role3);
user2.getRoles().add(role2);
user2.getRoles().add(role3);

role1.getUsers().add(user1);
role2.getUsers().add(user1);
role2.getUsers().add(user2);
role3.getUsers().add(user2);

session.save(user1);
session.save(user2);
session.save(role1);
session.save(role2);
session.save(role3);

transaction.commit();
}
```

在多对多的保存操作中，如果进行了双向维护关系，就必须有一方放弃外键维护权。一般由被动方放弃，用户主动选择角色，角色是被选择的，所以一般角色要回放弃外键维护权。但如果只进行单向维护关系，那么就不需要放弃外键维护权了。

将数据保存到数据库了以后，那么多对多会有哪些相关的操作呢？我们来学习一下多对多的相关的操作。



1.4.2 多对多的相关操作

1.4.2.1 级联保存或更新

之前已经学习过一对多的级联保存了，那么多对多也是一样的。如果只保存单独的一方是不可以的，还是需要保存双方的。如果就想保存一方就需要设置级联操作了。同样要看保存的主控方是哪一端，就需要在那一端进行配置。

【保存用户级联角色】

保存的主控方是用户，需要在用户一端配置：

```
<!-- 关联关系的映射的配置 -->
<!--
    set标签:
        name      :关联的另一方的集合的属性名称。
        table     :中间表的名称
-->
<set name="roles" table="sys_user_role" cascade="save-update">
<!--
    key标签:
        column    :当前对象在中间表中的外键的名称。
-->
<key column="user_id"/>
<!--
    many-to-many标签
        class     :关联另一方的类的全路径。
        column    :关联的另一方在中间表的外键名称
-->
<many-to-many class="cn.itcast.hibernate.demo2.Role" column="role_id"/>
</set>
```

配置好了级联了以后，就可以编写测试代码了。

```
@Test
/**
 * 多对多只保存一边 也是不可以的。
 * 配置级联:保存用户级联角色.在 User.hbm.xml 中<set>上配置 cascade="save-update"
 */
public void demo2() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();

    // 创建两个用户:
    User user1 = new User();
    user1.setUser_name("张三");

    User user2 = new User();
    user2.setUser_name("李四");
    // 创建三个角色:
    Role role1 = new Role();
```



```

role1.setRole_name("行政管理");
Role role2 = new Role();
role2.setRole_name("人事管理");
Role role3 = new Role();
role3.setRole_name("财务管理");

// 建立关系：如果建立了双向的关系 一定要有一方放弃外键维护权。
user1.getRoles().add(role1);
user1.getRoles().add(role3);

user2.getRoles().add(role2);
user2.getRoles().add(role3);

role1.getUsers().add(user1);
role2.getUsers().add(user2);

role3.getUsers().add(user1);
role3.getUsers().add(user2);

session.save(user1);
session.save(user2);
/*session.save(role1);
session.save(role2);
session.save(role3);
*/
tx.commit();
}

```

【保存角色级联用户】

保存的主控方是角色，就需要在角色端配置：

```

<set name="users" table="sys_user_role" cascade="save-update">
  <key column="role_id"/>
  <many-to-many class="cn.itcast.hibernate.demo2.User" column="user_id"/>
</set>

```

配置好了级联了以后，就可以编写测试代码了。

```

@Test
/**
 * 多对多只保存一边 也是不可以的.
 * 配置级联：保存角色 级联用户 在 Role.hbm.xml 中配置 cascade="save-update"
 */
public void demo3() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();

```



```
// 创建两个用户：
User user1 = new User();
user1.setUser_name("张三");

User user2 = new User();
user2.setUser_name("李四");
// 创建三个角色：
Role role1 = new Role();
role1.setRole_name("行政管理");
Role role2 = new Role();
role2.setRole_name("人事管理");
Role role3 = new Role();
role3.setRole_name("财务管理");

// 建立关系：如果建立了双向的关系 一定要有一方放弃外键维护权。
user1.getRoles().add(role1);
user1.getRoles().add(role3);

user2.getRoles().add(role2);
user2.getRoles().add(role3);

role1.getUsers().add(user1);
role2.getUsers().add(user2);

role3.getUsers().add(user1);
role3.getUsers().add(user2);

/*session.save(user1);
session.save(user2);*/
session.save(role1);
session.save(role2);
session.save(role3);

tx.commit();
}
```

级联保存说完了，那么我们来看下级联删除的操作，但是在多对多种级联删除是不会使用的，因为我们不会有这类的需求，比如删除用户，将用户关联的角色一起删除，或者删除角色的时候将用户删除掉。这是不合理的。但是级联删除的功能 Hibernate 已经提供了该功能。所以我们只需要了解即可。

1.4.2.2 级联删除：（了解）

【删除用户级联角色】



主控方是用户，所以需要在用户端配置：

```
<set name="roles" table="sys_user_role" cascade="delete">
  <!--
    key标签:
      column :当前对象在中间表中的外键的名称。
  -->
  <key column="user_id"/>
  <!--
    many-to-many标签
      class :关联另一方的类的全路径。
      column :关联的另一方在中间表的外键名称
  -->
  <many-to-many class="cn.itcast.hibernate.demo2.Role" column="role_id"/>
</set>
```

配置好了以后就可以编写代码了。

```
@Test
// 级联删除：删除用户 级联删除 角色。
public void demo4() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();

    User user = session.get(User.class, 11);
    session.delete(user);

    tx.commit();
}
```

这个时候我们发现用户被删除了，同时用户所关联的角色也一起被删除了。

【删除角色级联删除用户】

主控方是角色，所以需要在角色端配置：

```
<set name="users" table="sys_user_role" cascade="delete">
  <key column="role_id"/>
  <many-to-many class="cn.itcast.hibernate.demo2.User" column="user_id"/>
</set>
```

配置好了以后就可以编写代码了。

```
@Test
// 级联删除：删除角色 级联删除 用户。
public void demo5() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();

    Role role = session.get(Role.class, 31);
    session.delete(role);

    tx.commit();
}
```

这个时候我们发现角色被删除了，同时角色所关联的用户也一起被删除了。

多对多的级联我们已经介绍过了，但是级联并不是主要的操作，我们其实更多的会去使用多对



多的一些其他的操作，比如给某个用户选择一个新的角色，或者为某个用户改选已经选好的新角色，或者为某个用户移除某个角色。这些操作是我们以后开发中经常使用的。所以我们来学习下如何完成类似的相关的操作。

1.4.2.3 多对多的其他操作：

其实多对多的关系主要是靠中间表来维护的，那么在 Hibernate 中多对多主要是靠关联的集合来维护的，所以我们只需要关心如何操作性集合即可。那么我们来看下面的几个示例：

【删除某个用户的角色】

```
@Test
// 将 2 号用户中的 1 号角色去掉.
public void demo6() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();

    // 查询 2 号用户:
    User user = session.get(User.class, 21);
    // 查询 1 号角色
    Role role = session.get(Role.class, 11);

    // 操作集合 相当于 操作中间表.
    user.getRoles().remove(role);

    tx.commit();
}
```

【将某个用户的角色改选】

```
@Test
// 将 2 号用户中的 1 号角色去掉 改为 3 号角色
public void demo7() {
    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();

    // 查询 2 号用户:
    User user = session.get(User.class, 21);
    // 查询 1 号角色
    Role role1 = session.get(Role.class, 11);
    // 查询 3 号角色
    Role role3 = session.get(Role.class, 31);

    user.getRoles().remove(role1);
    user.getRoles().add(role3);

    tx.commit();
}
```



```
}
```

【给某个用户添加新角色】

```
@Test
// 给 1 号用户添加 2 号角色
public void demo8(){
    Session session = HibernateUtils.getCurrentSession();
    Transaction tx = session.beginTransaction();

    // 查询 1 号用户:
    User user = session.get(User.class, 11);
    // 查询 2 号角色:
    Role role = session.get(Role.class, 21);

    user.getRoles().add(role);
    tx.commit();
}
```

以上的这些操作是我们以后在多对多的关系中经常使用的操作，所以需要大家记住。